Exercise1 :

proof:

denote the original Array is A_old. denote A is the Array A_old after the loop.
n = |A_old|

consider the following Loop Invariant:
Inv := { (for all j such that i < j < n , we have A[j] = A_old[(j+1) mod n])
&& (x = A_old[(i+1) mod n])
&& (for all j such that 0 <= j <= i , we have A[j] = A_old[j]) }

denote (1) := (for all j such that i < j < n , we have A[j] = A_old[(j+1) mod n])
(2) := (x = A_old[(i+1) mod n])
(3) := (for all j such that 0 <= j <= i , we have A[j] = A_old[j])

that is Inv := { (1) && (2) && (3) }

now we will use Loop invariant thm:

1.Initialization : we will check that Inv holds at the start of the loop. we have A = A_old

for (1): since i = n - 1 . then doesn't exist j such that n-1 = i < j < n.
Since false can implies everything, so (1) holds.

for (2): x = A[0] = A_old[0] = A_old[n (mod n)] = A_old[(i+1) mod n]
so (2) holds.

for (3):since A = A_old .so for all j such that 0 <= j <= i , A[j] = A_old[j] holds.

Thus , Inv holds at the start of the loop.

2.Preservation: assuming {Inv && Cond} holds , we will check that Inv still holds after the loop
here Cond is i >= 0 .

for (1) : after the loop , i' = i - 1.
so we need to check that for all j such that i - 1 = i' < j < n , we have A[j] = A_old[(j+1) mod n].
since the Inv holds , so we just need to check the case j = i.
by the code:
A[j] = A[i] = x = A_old[i+1] = A_old[(j+1) mod n]
so (1) is still holds after the loop.

for (2) : after the loop , we have x' and i' = i -1 and A
so we need to check that x' = A_old[(i'+1) mod n]
by the code:
x' = A_old[i] = A[i] by Inv(3)
so A_old[i] = A[i] = A_old[(i'+1) mod n]
so (2) is still holds after the loop.

for (3) : after the loop , we have x' and i' = i -1 and A

```
            so we need to check that for all j such that 0 <= j <= i' = i-1 , we have A[j] = A_old[j]
            it is trivially true, because i-1 < i, by Inv(3) we can immediately get it.


    Thus Inv still holds after the loop.


3.Termination: The loop will terminate because i = n-1 is a natural number and i - 1 ....-1 it will < 0 at some time
               When the loop terminates, i = −1. Substituting this into Inv(1):
                  For all j such that −1 < j < n, we have A[j]=A_old[(j+1)(modn)].
                  Specifically, when j = n−1: A[n−1] = A_old[n(modn)] = A_old[0].
                  This matches the required condition B[n−1]=A[0]
               So the Cond will be false at some time, that is the loop will terminate.



    Therefore, by the Loop invariant thm , {Inv && !Cond} will holds at the end of the loop.
    that is  { (for all j such that i < j < n , we have A[j] = A_old[(j+1) mod n])
            && (x = A_old[(i+1) mod n])
            && (for all j such that 0 <= j <= i , we have A[j] = A_old[j])
            && i < 0}


    so
            >>> |A| = |A_old| is trivially true , because in the code we doesn't change the size of the Array.
            >>> in (1) we can take j = 0 => A[0] = A_old[1] = A_old[0+1]
                                  take j = n -1  => A[n-1] = A_old[n mod n] = A_old[0]
            so isLeftRotation holds. Q.E.D
```

Exercise2:

At the beginning of the Exercise , i want to verify a notation :
that is for some function f , f^k(n) is the same as (f(n))^k . it is just a notaiton.

Now , beginning the Exercises :

We have the definition of 'O' notation:
    We say that g belong to O(f)
    iff
    exist N0 in natural numbers , and c >0 ,
    such that for all n >= N0 , we have g(n) <= c*f(n).

(a)
    proof:
        our hypothesis are f , g are belong to O(h)

        by definition, that is :

        1. exist N1 in nature number , and c1 > 0,
        such that for all n >= N1 , we have f(n) <= c1*h(n)

    2. exist N2 in the nature number , and c2 > 0,
    such that for all n >= N2 , we have g(n) <= c2*h(n)

now , we claim two Lemmas:

Lemma 1 : if f , g are belong to O(h) , then f + g is also belong to O(h)

    PROOF OF LEMMA 1:
        take N := max{N1 , N2} , c := 2*max{c1 , c2}
        then for all n >= N , we have :
            f(n) + g(n) <= c1*h(n) + c2*h(n) <= 2*max{c1 , c2}*h(n) = c*h(n)

        Thus , f + g is also belong to O(h) , LEMMA 1 DONE!

Lemma 2 : if f is belong to O(h) , then f^k is belong to O(h^k) , for some k in the natural numbers.
    PROOF OF LEMMA 2:
        take N := N1 , c := (c1)^k
        then for all n >= N1 , we have:
            f(n) <= c1*h(n) implies f^k(n) <= (c1^k)*h^k(n) that is f^k(n) <= c*h^k(n)

        Thus f^k is belong to O(h^k) , LEMMA 2 DONE!

Now , back to the Exercise(a) , by Lemma 1 and Lemma 2 we immediately get that :
(f(n) +g(n))^k is belong to O(h^k(n)) . Q.E.D

(b)

proof:

we have the hypothesis : f is belong to O(n) , that is :

exist N1 in nature number , c > 0 ,

such that for all n >= N1 , we have f(n) <= c*n .

now we denote g(n) := SUM(i from 1 to n)f^2(i) , our aim is to prove that g is belong to O(n^3).

now , since f is monotonically non-decreasing function ,

so for all 1 <= i <= n, we have f(i)≤f(n)

g(1) <= g(n) , g(2) <= g(n) ,..., g(n-1) <= g(n)

so g(n) <= n*f^2(n) .

now it is enough to prove that n*f^2(n) is belong to O(n^3)

take N := N1 , C := c^2

by hypothesis , we have for all n >= N = N1 , f(n) <= c*n implies f^2(n) <= (c^2)*(n^2) impies n*f^2(n) <= n*(c^2)*(n^2) = c^2(n^3) = C*(n^3)

Therefore , we proved that n*f^2(n) is belong to O(n^3). implies that g(n) is belong to O(n^3) Q.E.D

(c)

proof:

Hypothesis: f is belong to O(g^2) and f is also belong to O(h^2)

that is :

1. exist N1 in nature number , c1 > 0,

such that for all n >= N1 , we have f(n) <= c1*g^2(n)

2.exist N2 in nature number , c2 > 0,

such that for all n >= N2 , we have f(n) <= c2*h^2(n)

Now take N:= max{N1 , N2} , c := square root of (c1*c2)

by hypothesis for all n >= N , we have f(n)*f(n) <= (c1*g^2(n))*(c2*h^2(n))

now take the square root both side (f,g,h >= 1), we get that f(n) <= c *(g(n)*h((n)))

That is f is belong to O(g*h) Q.E.D

Exercise3:

(a)

the Definition of Node is:

```
class Node<Integer>{
    Integer element;
    Node<Integer> left;
    Node<Integer> right;
}
```

our Algorithm is as follows:

```
public int Sum_of_min_and_max_of_the_tree(Node<Integer> root){

    if (root == null ){
        return 0;
    }

    Node<Integer> minimal = root;

    Node<Integer> maximal = root;

    while(minimal.left != null){
        minimal = minimal.left;
    }
```

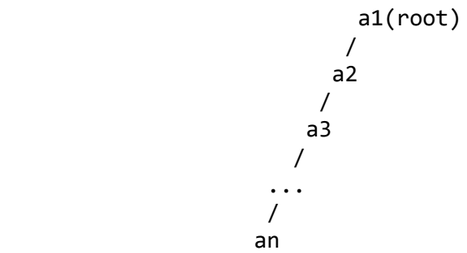
```
        while(maximal.right != null){
            maximal = maximal.right;
        }

        return (minimal.element + maximal.element);
    }
```

==========

(b)

The worst case:

in my code , if we define the binary seach tree Wn is like this(each node only have left child , no right child)(skewed tree):

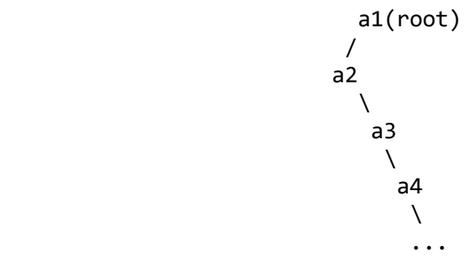


then , in the first whlie loop , we need to do n times , that is time complexity is bounded by n (O(n)).
the second while loop takes 1 step , since root.right == null ,
T(n)=0(n)+0(1)=0(n)
so the worst case is 0(n)

(c)

The best case:

if We define the binary search tree Bn is like this(the root has only left child , no right child):



\
an

then for two while loops in the code , each while loop just need to take only one step to find the minmal and the maximal, that is a2 and a1.

so in the best case , the time complexity is 0(1).

Exercise 4:

(a)

the Definition of Node is:

```
=====================================================================

public class Node<Integer>{
    Integer value;
    Node<Integer> left;
    Node<Integer> right;
}

=====================================================================
```

our Algorithm is :

```
=============================================================================================================================================
```

```
private int compute_size_of_tree(Node<Integer> root){
    Stack<Node<Integer>> stack = new Stack<>();
    stack.push(root);
    int res = 0;
    while (stack.size() > 0) {
        Node<Integer> subtree = stack.pop();
        if (subtree != null) {
            res ++;
            stack.push(subtree.left);
            stack.push(subtree.right);
        }
    }
    return res;
}


public ArrayList<Integer> output_sorted_array_of_two_trees(Node<Integer> root1 , Node<Integer> root2){

    assert root1 != null && root2 != null : "tree can't be empty";

    ArrayList<Integer> result_array = new ArrayList<>();

    Node<Integer> smaller = new Node<>();
    Node<Integer> bigger = new Node<>();

    if( compute_size_of_tree(root1) >= compute_size_of_tree(root2) ){
        smaller = root2;
        bigger = root1;
    }
    else{
        smaller = root1;
        bigger = root2;
    }

    Stack<Node<Integer>> stack = new Stack<>();
    stack.push(smaller);
    int res = 0;
    while(stack.size() > 0){
        Node<Integer> subtree = stack.pop();
        if(subtree != null){
            if( subtree.value.search_in_tree(bigger) ){
                result_array.add(subtree.value);
            }
        }
    }

    return Collection.sort(result_array);
}

COMMENT:


in my code , we have private method 1 :  "compute_size_of_tree" , in the lecture ,we have seen that the time complexity of it is O(n) in the worst case.
```

method 2 : "output_sorted_array_of_two_trees" , in this method , we determine the smaller tree of two trees
and for each element in the smaller one , we search it whether it is in the other bigger one,
if is, then add it to our result array. since the while loop is the same structure of the method 1 ,
and in the lecture we learn that in AVL trees , the search method is O(logn) in the worst case
therefore , in our Algorithm , the worst time complexity is O(n*logn).

==========

(b)

The definition of the Node ommit

our Algorithm is:

==========

```
public int sum_of_the_value_in_interval(int l, int r, Node<Integer> A) {
    return inorderSum(A, l, r);
}

private int inorderSum(Node<Integer> node, int l, int r) {

    if (node == null) return 0;

    int sum = 0;

    if (node.value > l) {
        sum += inorderSum(node.left, l, r);
    }

    if (node.value >= l && node.value <= r) {
        sum += node.value;
    }

    if (node.value < r) {
        sum += inorderSum(node.right, l, r);
    }

    return sum;

}
```

COMMENT:

    Each node is visited at most once, and only the nodes whose values fall within the interval, along with some nodes on the paths from the root to those nodes, are visited.
    Assuming the height of the binary tree is O(log n) (property of an AVL tree), and let m be the number of nodes actually within the interval (m ≤ r−l+1), then the total number of visited nodes is O(log n + m).
    Therefore, the time complexity is O(log n + (r−l))

==========

(c)

    our data structure is:

==========

```
    public class find_PreImage{
        private ArrayList<Integer> sorted_array = new ArrayList<>();
        private int n;
        public find_PreImage (function f , int n){
            assert n >= 1 : "n must in the natural number";
            this.n = n;
            for( int i = 1 ; i < n+1 ; i++ ){              //at the end of the for loop , we will get a sorted array , because function f is strictly increasing. and this take almost O(n).
                sorted_array.add(f(i));
            }
        }

        public int output_pre_image(int j){
            //we will do binary search
            int low = 0 , high = n-1;
            while(low <= high){
```

```
                int mid = low + (high - low)/2;
                if( sorted_array.get(mid).equals(j) ){
                    return mid + 1;
                }
                else{
                    if(sorted_array.get(mid) < j){
                        low = mid + 1;
                    }
                    else{
                        high = mid - 1;
                    }
                }
            }

            return -1;         //actually we will not reach here, because array is sorted and use binary search will find the value
        }


    }
```

COMMENT: in our code , we are doing initailzation in the construcor , and doing the main algotithm in the method"output_pre_image" , the constructor take O(n) and the operation is binary search , take O(logn).

==========

Exercise 5:

(a)
proof:

by the definition of ternary trees , we can get that a perfect banlance ternary tree is like this:

so we get the table :

h	number of nodes
1	1 = 3^0
2	1 + 3 = 3^0 + 3^1
3	1 + 3 + 9 = 3^0 + 3^1 + 3^2
4	1 + 3 + 9 + 27 = 3^0 + 3^1 + 3^2 + 3^3
.	
.	
.	
n	3^0 + 3^1 + 3^2 + 3^3 + ... + 3^(n-1)

Thus , we claim that NB(h) = 3^0 + 3^1 + 3^2 + 3^3 + ... + 3^(h-1) ========= (3^h - 1)/2
(by math)

Now , we will prove it by induction:

1.Base case: when h = 1 . NB(1) = 1 = (3^1 - 1)/2

2.Inductive step : suppose that this formula is holds for k , will prove the k+1 case.
so Inductive Hypothesis is NB(k) = (3^k - 1)/2
now , by the definition of perfect balance trees : NB(k+1) = NB(k) + 3*(3^(k-1)) = (3^k - 1)/2 + 3*(3^(k-1))
= (3^k - 1)/2 + 3^k
= (3^(k+1) - 1)/2

so NB(k+1) = (3^(k+1) - 1)/2
so it is also holds for k+1 case.

Therefore , by inductive principle , NB(h) = (3^h - 1)/2 Q.E.D

(b)
proof:

    note that n <= NB(h) for all h in the natural numbers.

    so n <= (3^h - 1)/2 <= (0.5)*3^h

    now (0.5)*3^h belongs to O(3^h) , because we can take N := 1 , c := 1 ,
    then it is trivially true that for all h >= N , (0.5)*3^h <= c*3^h

    so n is belong to O(3^h).   Q.E.D